

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในโครงงาน

ในหัวข้อนี้จะกล่าวถึงทฤษฎีพื้นฐานต่างๆ ที่เกี่ยวข้องในการวิจัย และพื้นฐานของระบบการจำแนกรหัสแท่งสองมิติ ชนิด ดาต้าแมทริกซ์ (Datamatrix) ด้วยการประมวลผลภาพ ซึ่งเนื้อหาในบทนี้จะกล่าวถึง การประมวลผลภาพ หลักการและโครงสร้างของรหัสแท่ง จาวาคืออะไรและสามารถทำอะไรได้บ้าง และพื้นฐานของเว็บเซอร์วิส ซึ่งเนื้อหาทั้งหมดนี้มีความจำเป็นสำหรับการศึกษาและประเมินประสิทธิภาพของระบบการจำแนกรหัสแท่งด้วยการประมวลผลภาพ

2.1 การประมวลผลภาพ (Image Processing)

การประมวลผลเชิงตัวเลข หมายถึง การนำภาพทั่วไปมาประมวลผลด้วยเครื่องคอมพิวเตอร์ โดยภาพที่นำมาประมวลผลด้วยเครื่องคอมพิวเตอร์นี้จะถูกแทนที่ด้วยตัวเลขให้อยู่ในรูปของเมตริกซ์ แต่ภาพที่ได้โดยส่วนมากแล้วจะเป็นภาพที่ได้จากตัวรับสัญญาณซึ่งอยู่ในรูปของฟังก์ชัน $f(x,y)$ ที่ต่อเนื่องในระนาบสองมิติ (คือแกน X และ Y) โดยจะเป็นสัดส่วนกับความสว่างหรือความเข้มของภาพที่ตำแหน่งซึ่งเรียกว่าระดับสีเทา (gray level)

2.1.1 การแทนภาพด้วยข้อมูลแบบดิจิทัล

ภาพข้อมูลแบบดิจิทัล (Digital image) เป็นภาพที่ถูกแปลงมาจากอนาล็อก (Analog) ให้อยู่ในรูปของตัวเลขโดยภาพอนาล็อก ถูกแบ่งเป็นพื้นที่สี่เหลี่ยมเล็กๆ เรียกว่า พิกเซล (Pixel) ในแต่ละพิกเซลจะถูกระบุตำแหน่งโดย (x,y) และค่าระดับสีเทาของพิกเซล โดยเราสามารถแปลงข้อมูลไฟล์ภาพเป็นข้อมูลแบบดิจิทัลได้ โดยมีขั้นตอนและวิธีดังนี้

เมื่อนำสัญญาณอนาล็อกที่ต้องการประมวลผล มาผ่านส่วนที่เรียกว่า ดิจิไทเซอร์ (Digitizer) ซึ่งจะมีหน้าที่ในการเปลี่ยนแปลงสัญญาณอนาล็อกให้เป็นสัญญาณดิจิทัล อุปกรณ์ส่วนนี้ได้แก่ กล้องโทรทรรศน์ดิจิไทเซอร์ จากนั้นทำการควอนไทซ์ (Quantizing) เพื่อที่จะประมวลสัญญาณด้วยระบบคอมพิวเตอร์ ฟังก์ชันของภาพ $f(x,y)$ จะถูกทำให้เป็นสัญญาณไม่ต่อเนื่องในระนาบของภาพ ซึ่งเราเรียกว่าการสุ่มภาพ (image sampling) ของฟังก์ชันที่ได้เรียกว่า การควอนไทซ์ระดับสีเทา (Gray Level Quantization) ก็จะได้ข้อมูลที่เป็นดิจิทัล

สมมติว่าสัญญาณภาพต่อเนื่อง $f(x,y)$ ถูกดิจิไทเซอร์ในระนาบ x และ y เป็นช่วงเท่าๆกันเราสามารถจัด $f(x,y)$ ให้อยู่ในรูปของเมตริกซ์ขนาด $n*n$ ได้ดังสมการที่ 2.1

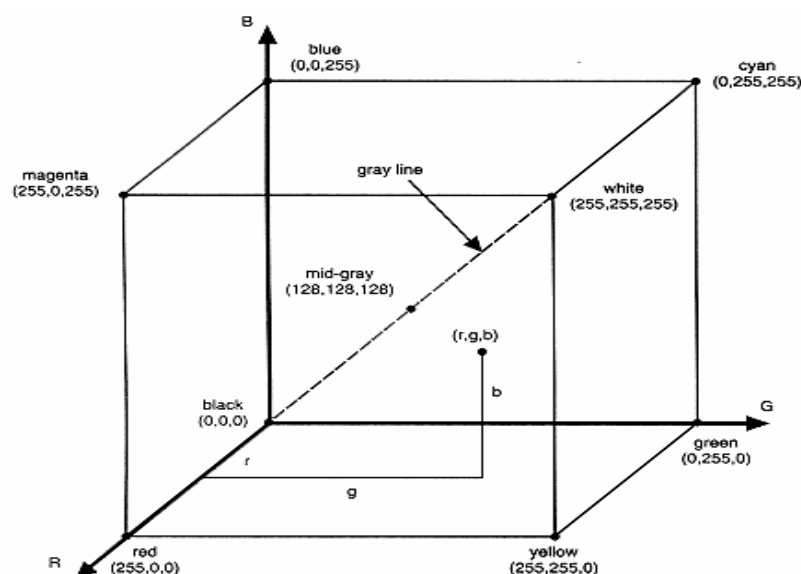
$$\begin{array}{cccccc}
 f(0,0) & f(0,1) & f(0,2) & \dots\dots\dots & f(0,N-1) \\
 f(1,0) & f(1,1) & f(1,2) & \dots\dots\dots & f(1,N-1) \\
 \vdots & \vdots & \vdots & & \vdots \\
 f(N-1,0) & f(N-1,1) & f(N-1,2) & \dots\dots\dots & f(N-1,N-1)
 \end{array}
 \quad (2.1)$$

ซึ่งสมการที่ 2.1 ค่าแต่ละค่าที่อยู่ในเมทริกซ์จะเรียกว่าอิลิเมนต์ของภาพ (Picture Element) จากกระบวนการสร้างภาพดิจิทัลข้างต้น จะเห็นได้ว่าเราสามารถทราบขนาดความละเอียดของภาพ ขนาด $N \times N$ พิกเซล และจำนวนระดับสีเทา ในทางปฏิบัติการทำคอนไต์เซชันในระบบภาพดิจิทัล จะมีค่าดังสมการที่ 2.2

$$\begin{array}{ll}
 B & = \quad N \times N \times M \text{ บิต} \\
 \text{เมื่อ } B & = \quad \text{ขนาดของข้อมูลที่เป็นดิจิทัล} \\
 G & = \quad \text{จำนวนของเกรย์สเกลที่ต้องการใช้ในการเก็บข้อมูลภาพ} \quad (2.2) \\
 M & = \quad \text{จำนวนบิตที่ใช้ในการแทนข้อมูลภาพ 1 พิกเซล} \\
 \text{โดยที่ } M & \text{สามารถหาได้จาก } G = 2^M
 \end{array}$$

2.1.2 โมเดลสี RGB และการแทนสี

สำหรับโมเดลสีที่นำมาใช้พิจารณาหาสีที่ใกล้เคียงที่สุดคือโมเดลสี RGB ซึ่งสีแต่ละสีในโมเดล RGB นี้จะประกอบไปด้วยสีหลัก 3 สี คือ สีแดง สีเขียว และสีน้ำเงินในอัตราส่วนที่ต่างกันไป โดยสีหลักแต่ละสีจะมีค่าเป็นเลขจำนวนเต็มตั้งแต่ 0 ถึง 255 ดังนั้นเราจึงสามารถนำรูปลูกบาศก์มาใช้แทนโมเดลสี RGB ได้



รูปที่ 2.1 โมเดลสี RGB

ในแต่ละแมสสีมีขนาด 8 บิต ซึ่งทั้ง 3 สีรวมกันก็จะมีขนาดเท่ากับ 24 บิต จึงสามารถสร้างสีใหม่ได้ถึง $256 \times 256 \times 256$ เท่ากับ 16,777,216 สี ดังนั้นภาพสีขนาด 24 บิต จะมีค่าสีของในแต่ละพิกเซลอยู่ในช่วงที่ประกอบด้วย

R, G, B ที่ระดับ 0 จนถึง 255 โดยที่ $(0 \leq R, G, B \leq 255)$

สามารถแปลงโมเดลสี ให้เป็น Gray Scale ได้โดยใช้สมการที่ 2.3

$$\text{Gray Scale} = 0.299 * R + 0.587 * G + 0.114 * B \quad (2.3)$$

แต่เราสามารถใช้อีกสมการ โดยการหาค่าเฉลี่ยทั้งสามสีดังสมการที่ 2.4

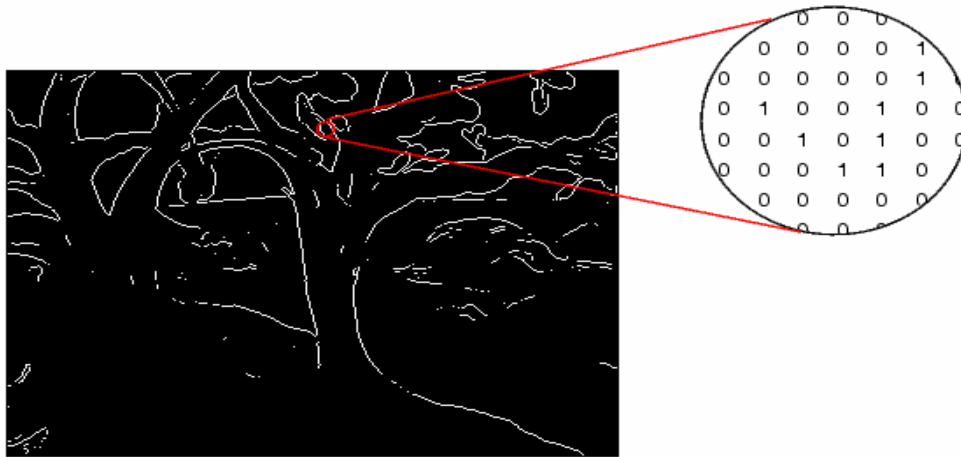
$$\text{Gray Scale} = (R+G+B)/3 \quad (2.4)$$

จากรูปที่ 2.1 ค่า Gray Scale ก็คือค่าที่อยู่ในช่วง (0, 0, 0) จนถึง (1, 1, 1)

2.1.3 การสร้างภาพไบนารี (Binary Image)

อุปกรณ์ที่มีความสามารถในการแสดงผลได้แค่ 2 ระดับ หรือ 2 สี คือสีขาวกับสีดำยังมีการใช้กันอย่างแพร่หลาย เนื่องจากมีราคาถูก ดังนั้นการที่จะแสดงผลหรือพิมพ์รูปภาพที่มีระดับความเข้มของภาพหลายระดับความเข้มของภาพหลายระดับ ซึ่งมีความสามารถในการแสดงผลของอุปกรณ์เหล่านั้นที่มีเพียงแค่ 2 ระดับเท่านั้น

จะเห็นได้ว่าการแก้ปัญหาการแสดงผลภาพที่มีความเข้มหลายระดับบนอุปกรณ์ที่สามารถแสดงผลได้ 2 ระดับนั้น จะต้องทำการแปลงข้อมูลภาพที่ต้องการให้เป็นภาพไบนารีก่อน ซึ่งการสร้างภาพไบนารีนี้นั้นก็หมายถึงการแปลงข้อมูลภาพที่มีระดับความเข้ม (Multi Level Image) ให้เป็นภาพที่มีระดับความเข้มเพียง 2 ระดับ คือ 1 จุดภาพมีได้ 2 ค่าเท่านั้น คือ 0 กับ 1 โดยจุดภาพที่แทนด้วย 0 จะหมายถึงจุดภาพที่มีสีดำ ส่วนจุดที่แทนด้วย 1 จะหมายถึงจุดภาพที่แทนด้วยสีขาว เมื่อทำการแปลงภาพไบนารีแล้วจึงนำภาพนั้นไปแสดงผลบนอุปกรณ์เหล่านั้น จะเห็นได้ว่าการแปลงข้อมูลภาพหลายระดับให้เป็นภาพไบนารีจึงมีความจำเป็นและมีประโยชน์อย่างมากในการแสดงผลภาพที่มีระดับความเข้มของภาพหลายระดับบนอุปกรณ์ที่มีความสามารถในการแสดงผล 2 ระดับ สำหรับประโยชน์อีกประการหนึ่งของการแปลงภาพเป็นภาพไบนารีคือ เป็นการลดเนื้อที่ในการเก็บข้อมูลภาพทั่วไปที่ใช้ในการเก็บถึง 8 บิต แต่เมื่อสร้างเป็นภาพไบนารีแล้วสามารถลดได้ถึง 8 เท่า นั่นคือ 1 จุดภาพจะใช้เนื้อที่ในการเก็บ 1 บิตเท่านั้น ทั้งยังสามารถนำไปประยุกต์ใช้ในงานที่เกี่ยวข้องกับการประมวลผลภาพได้อย่างมากมาย



Pixels in a Binary Image Have Two Possible Values: 0 or 1

รูปที่ 2.2 ภาพไบนารีและข้อมูลของแต่ละพิกเซล

ในการสร้างภาพไบนารีสามารถทำได้โดยใช้เทคนิคการทำเทรชโฮล (Threshold) โดยพิจารณาว่าจุดภาพใดควรจะเป็นจุดขาวหรือจุดดำ จะทำโดยการเปรียบเทียบระหว่างจุดภาพเริ่มต้นกับค่าคงที่หนึ่งๆที่เรียกว่า Threshold Value เทคนิคนี้ใช้กันมากในกรณีที่มีข้อมูลภาพมีลักษณะที่แตกต่างกันระหว่างวัตถุ (Object) และพื้นหลัง (Background) โดยค่าของจุดภาพใดๆ ที่มีค่าน้อยกว่าค่าเทรชโฮล จะถูกกำหนดให้เป็น 0 (จุดดำ) และถ้าค่าของจุดภาพใด ๆ ที่มีค่ามากกว่า หรือเท่ากับค่าเทรชโฮล ก็จะถูกเปลี่ยนให้เป็น 1 (จุดขาว) ซึ่งการทำงานสามารถแสดงได้โดยสมการนี้

$$\begin{aligned}
 b(x,y) &= 0 ; g(x,y) < \text{Ther} \\
 &= 1 ; g(x,y) \geq \text{Ther} \\
 b(x,y) &\text{ ข้อมูลภาพผลลัพธ์เป็นไบนารี} \\
 g(x,y) &\text{ ข้อมูลภาพมีระดับความเข้ม 0 ถึง L ระดับ} \\
 \text{Ther} &\text{ ค่าเทรชโฮลเป็นค่าคงที่ที่อยู่ระหว่าง 0 ถึง L ระดับ} \\
 0 &\text{ จุดดำ} \\
 1 &\text{ จุดขาว} \\
 \text{โดยที่ } L &\text{ คือระดับความเข้มของจุดภาพ}
 \end{aligned}
 \tag{2.5}$$

ในการสร้างภาพไบนารีโดยใช้เทคนิคเทรชโฮล เพื่อให้ได้ผลลัพธ์ที่เหมาะสมและคมชัด สิ่งที่สำคัญที่สุดคือ ค่าเทรชโฮล เนื่องจากถ้าเลือกค่าเทรชโฮล ที่ไม่เหมาะสม (ค่า เทรชโฮล ที่มีค่ามากหรือน้อยเกินไป) ภาพที่ได้อาจจะไม่เหมาะสม ขาดความคมชัดและรายละเอียดบางส่วนหายไป

กล่าวคือภาพที่ได้อาจจะมืดเกินไป (จุดดำมากเกินไป) หรือสว่างเกินไป (จุดขาวมากเกินไป) หรือภาพที่ได้มีสิ่งรบกวน (Noise) เกิดขึ้น ทำให้ภาพที่ได้มานี้ไม่ดีเท่าที่ควร ดังนั้นปัญหาของการสร้างภาพโดยวิธีเทอร์ซโซล นี่คือการหาวิธีที่จะคำนวณหาเทอร์ซโซล ที่เหมาะสมสำหรับแต่ละภาพที่จะนำมาสร้างภาพไบนารี ซึ่งมีวิธีการคำนวณหาเทอร์ซโซล หลายวิธี โดยแต่ละวิธีจะเหมาะสมกับลักษณะงานที่แตกต่างกันไป เช่น การหาโดยวิธีการกำหนดค่าล่วงหน้า (การกำหนดค่าเอาเองโดยผู้ใช้) การหาค่าเทอร์ซโซล โดยการกำหนดค่ากลาง (การกำหนดโดยการหาค่าเฉลี่ยแบบอัตโนมัติ)

2.1.4 กระบวนการรอบๆจุดพิกเซล (Area Processing)

การทำฟิลเตอร์ Low pass Filter

เป็นการกรองให้ภาพราบเรียบ หรือการเกลี่ยภาพจะใช้วิธีการทำให้ภาพเบลอ และเป็นการลดสัญญาณรบกวนภายในภาพด้วย ซึ่งการทำให้ภาพเบลอจะเป็นการนำรายละเอียดเล็กๆ น้อยๆ ที่ไม่ต้องการในภาพออก และยังสามารถเกลี่ยรอยช่องว่างเล็กๆในเส้นตรง และเส้นโค้งในภาพได้ด้วยการกรองแบบ Low pass Filter จะทำโดยการแทนค่าของทุกๆ พิกเซลในภาพด้วยค่าเฉลี่ยของระดับสีเทาของบริเวณใกล้เคียง ซึ่งจะกำหนดโดยหน้าต่างของการกรอง ผลของกระบวนการนี้จะได้เป็นรูปภาพที่มีการลดการเปลี่ยนแปลงในรูปภาพซึ่งเป็นระดับของสีเทา ให้มีสัญญาณรบกวนน้อยลง ตัวอย่าง เช่นเส้นขอบของรูปภาพคือลักษณะของการเปลี่ยนแปลงในรูปภาพที่เป็นระดับสีเทา ดังนั้นการทำการกรองแบบเฉลี่ยจะมีผลกระทบทำให้เส้นขอบของรูปภาพนั้นหายไป ด้วย ทั้งนี้เนื่องมาจากการใช้จำนวนระดับของสีเทาที่ไม่เพียงพอนั่นเอง

ประโยชน์หลักของการใช้การกรองแบบเฉลี่ย คือ การลดรายละเอียดที่ไม่ตรงประเด็นในภาพต้นแบบออกไป โดยกระบวนการนี้จะเอาค่าบริเวณรอบๆพิกเซลมาคำนวณแล้วนำมาแทนค่าพิกเซลเดิม บริเวณพิกเซลรอบๆ นี้เรียกว่า Mask ส่วนในการคำนวณบริเวณรอบๆเรียกว่า การทำ Convolution

$$\frac{1}{9} \times$$

Pixel Value		
0	25	22
20	30	80
55	50	10

รูปที่ 2.3 การทำ Convolution ที่ Mask ขนาด 3×3

จากรูปที่ 2.3 จะแสดงการทำ Convolution กระบวนการนี้จะนำค่าบริเวณรอบๆ มาคำนวณ ซึ่งเป็นการกรองแบบเฉลี่ยที่ Mask ขนาด 3×3 โดยการคูณค่าสัมประสิทธิ์ ซึ่งเท่ากับ $1/9$ กับค่าผลรวมภายในเพื่อหาค่าพิกเซลใหม่ตรงกลางมาแทนค่าพิกเซลเดิม ซึ่งจะทำให้ผลลัพธ์ที่ได้ออกมานั้นมีประสิทธิภาพเพิ่มขึ้น

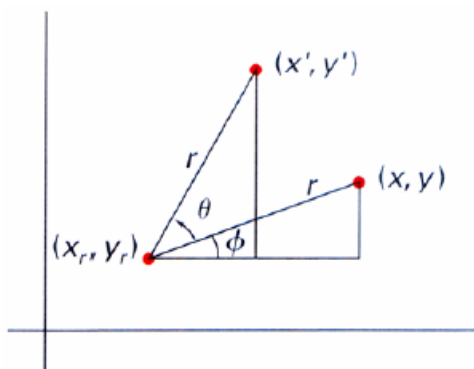
2.1.5 การหมุนวัตถุ (Rotation)

เป็นวิธีการเปลี่ยนแปลงพิกัดของภาพจากพิกัดใดๆ ให้อยู่ในพิกัดที่ต้องการภาพโดยการหมุนวัตถุ ในกรณีที่ Pivot point เป็นจุดใดๆ บนระนาบ (x, y) ทำได้โดย

1. เลื่อนจุดที่ต้องการหมุนไปยังจุดกำเนิดก่อน
2. ทำการหมุนโดยใช้ความสัมพันธ์การหมุน ซึ่งมีจุดศูนย์กลางที่จุดกำเนิด
3. เลื่อนจุดที่หมุนเรียบร้อยแล้วมาที่ตำแหน่ง Pivot point

ซึ่งเราสามารถแสดงได้ด้วยสมการต่อไปนี้

$$\begin{aligned} x' &= x_r + (x - x_r) \cos \theta - (y - y_r) \sin \theta \\ y' &= y_r + (x - x_r) \sin \theta + (y - y_r) \cos \theta \end{aligned} \quad (2.6)$$



รูปที่ 2.4 แสดงลักษณะการหมุนของวัตถุ

หรือสมการในรูปของ Matrix

$$\begin{aligned} \mathbf{P}' &= \mathbf{T}_r + \mathbf{R} \cdot (\mathbf{P} - \mathbf{T}_r) \\ \mathbf{R} &= \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \\ \mathbf{T}_r &= \begin{bmatrix} x_r \\ y_r \end{bmatrix} \end{aligned} \quad (2.7)$$

การทำ Transformation ด้วยสมการข้างต้น หลายครั้งจะทำให้เกิดข้อผิดพลาดสะสม สำหรับ Integer Arithmetic ดังนั้นจึงจำเป็นต้องใช้หลักการของวิธีนี้ คือจัด Geometric Transformation ในรูปของการคูณกันของ Matrix ซึ่งทำได้โดยการขยายพจน์ที่เป็น Matrix ขนาด 2×2 เป็น 3×3 และขนาด 2×1 เป็น 3×1 ตามลำดับ ซึ่งทำได้โดยจัดพิกัด Cartesian (x, y) ในรูปของ Homogeneous Coordinates (X_h, Y_h, h) ดังสมการที่ 2.8

$$x = \frac{x_h}{h} \quad y = \frac{y_h}{h} \quad (2.8)$$

ดังนั้นพิกัด Homogeneous อาจเขียนได้ใหม่เป็น ($x \times h, y \times h, h$) โดยทั่วไปเราสามารถเลือกค่า h เป็นจำนวนจริงบวกใดๆ แต่เพื่อความสะดวกมักจะเลือกให้ $h = 1$ ซึ่งจะได้พิกัดเป็น Homogeneous (x, y, 1)

โดยการใช้พิกัด Homogeneous (ตัวดำเนินการบนตัวแปรที่ตำแหน่ง a จะให้ผลเช่นเดียวกันกับตัวแปรที่ตำแหน่ง b) เราสามารถจัดรูป Transformation ใหม่ได้ดังสมการที่ 2.9

Rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2.9)$$

2.2 หลักการและโครงสร้างของรหัสแท่ง

2.2.1 หลักการทั่วไปของรหัสแท่ง

รหัสแท่ง หรือ บาร์โค้ด หมายถึง ระบบสัญลักษณ์ หรือเครื่องหมายประจำตัวซึ่งใช้แทนเลขรหัส โดยทั่วไปจะเป็นภาษาสากลที่ใช้เพื่อสื่อ หรือบ่งบอกถึงประเทศผู้ผลิต บริษัทที่ผลิตสินค้า ชนิดและราคาของ สินค้า เพื่อให้เกิดความสะดวกแก่ผู้ผลิตและผู้ประกอบการในการตรวจสอบสินค้า ตั้งแต่ขั้นตอนในการผลิต การเก็บสินค้า จัดจำหน่าย กำหนดนโยบายทางการตลาดรวมทั้งการเปลี่ยนแปลงราคาสินค้า ซึ่งสามารถประหยัดเวลาและค่าใช้จ่ายเป็นอย่างมาก เนื่องจากไม่ต้อง

จ้างแรงงานเพิ่มในการเปลี่ยนแปลงป้ายราคา และลดการจ่ายค่าจ้างแรงงานเพิ่ม การใช้รหัสแท่งก็ไม่ยากเพียงแต่นำเอาตัวเลขหรือรหัสข้อมูลที่ต้องการซึ่งได้กำหนดขึ้นมาเพื่อใช้ในการอ้างอิงข้อมูลของผลิตภัณฑ์นั้นๆ มาแปลงเป็นสัญลักษณ์แท่งดำสลับขาว ที่มีขนาดต่างกันตามขนาดของข้อมูลที่ต้องการอ้างอิงด้วยคอมพิวเตอร์ แล้วพิมพ์ติดบนตัวสินค้าตามต้องการ

จุดประสงค์ของรหัสแท่งที่ถูกพัฒนาขึ้นมาใหม่นั้นมีอยู่หลายประการด้วยกันเช่น

1. เพื่อให้สามารถแทนรหัสได้มากขึ้น
2. เพื่อให้เหมาะสมกับการใช้งานเฉพาะด้าน
3. เพื่อให้มีความหนาแน่นของรหัสต่อพื้นที่มากขึ้น

2.2.2 ความหลากหลายของรหัสแท่ง

การคิดรหัสแท่งของสินค้านั้นๆ โดยเฉพาะนอกจากจะคำนึงถึงความสะดวกรวดเร็วในการทำงานแล้วยังจะต้องคำนึงถึงการใช้มาตรฐานการกำหนดเลขหมายที่ได้รับการยอมรับจากทั่วโลกอีกด้วย

นอกจากนี้ รหัสยังมีลักษณะอื่นๆ ที่แตกต่างกันอีก เช่น เป็นรหัสแทนตัวเลข หรือรหัสแทนทั้งตัวเลขและตัวอักษร ความยาวของแถบรหัสคงที่หรือแปรเปลี่ยนได้เป็นต้น การเลือกใช้นั้นก็ขึ้นอยู่กับลักษณะงาน โดยเราจะพิจารณาเลือกรหัสจากชุดตัวอักษรที่รหัสสามารถแทนได้ ความยากง่ายในการใส่รหัส ความแม่นยำของรหัส ความยืดหยุ่นต่อความเร็วที่ใช้ในการอ่านและความต้านทานต่อความไม่สมบูรณ์ในการพิมพ์ อย่างไรก็ตามรหัสที่ใช้กันแพร่หลายในปัจจุบันได้แก่

รหัสแบบ 1 มิติ ได้แก่ UPC (Universal Product Code), EAN (European Article number), Code bar, "2 ใน 5" รหัส 39 ฯลฯ

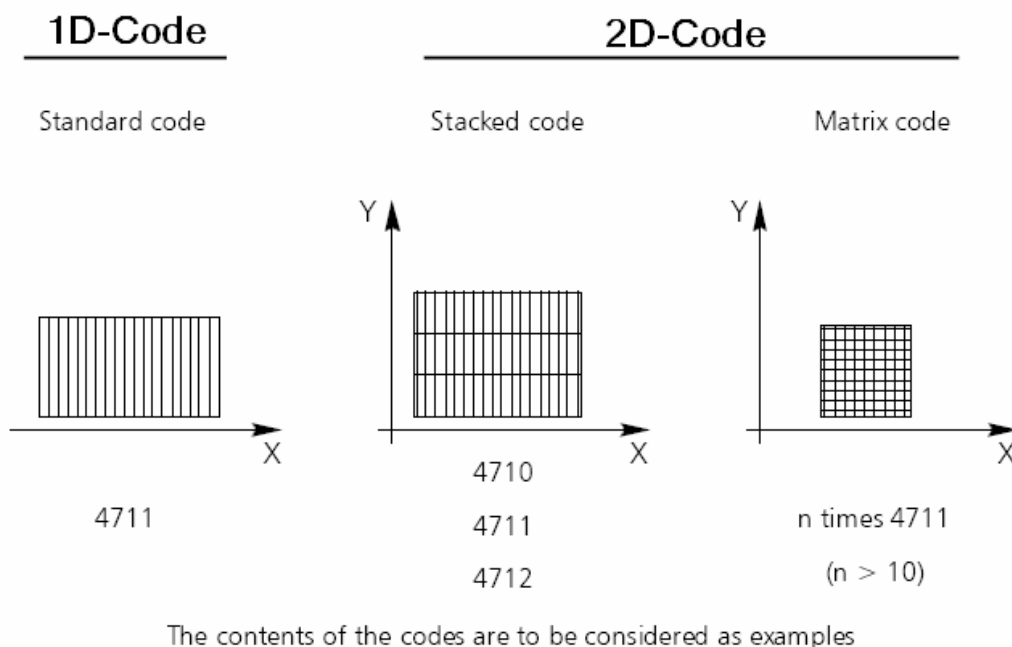
รหัสแบบ Stacked barcodes ได้แก่ Codablock, Code 16K, Code 49, PDF417, Micro PDF417

รหัสแบบ 2 มิติ ได้แก่ Data Matrix , PDF 417 , Maxi Code , QR Code, Aztec Code, SmartCode, Semacode ฯลฯ

2.2.3 ลักษณะสำคัญของรหัสแท่ง

แม้ว่าการสร้างและพัฒนารหัสแท่งได้ทำมาเป็นเวลานานแล้วก็ตามแต่สิ่งหนึ่งที่ยังคงเป็นเอกลักษณ์หรือโครงสร้างของรหัสที่ยังไม่เคยเปลี่ยน คือ ตัวรหัสจะประกอบไปด้วยรหัสที่เป็นแท่งสีดำสลับกับแถบขาว หรือ สีอื่นๆ สลับกัน โดยหลักการแล้วสีที่ใช้จะแบ่งเป็นสองลักษณะ คือ สีที่เข้มมากกว่าและสีที่เข้มน้อยกว่า

รหัสแท่งเป็นรหัสที่มีการสร้างและพัฒนาขึ้นมานานพอสมควรแล้วโดยการสร้างและพัฒนาของรหัสแท่งจะเป็นในรูปแบบที่มีชนิดของรหัสมากขึ้น ในปัจจุบันรหัสแท่งได้ถูกแบ่งออกเป็นสองรูปแบบหลักๆด้วยกันตามลักษณะโครงสร้างของมันคือ แบบ 1 มิติ (1D-Barcode) และแบบ 2 มิติ (2D-Barcode) โดยดูได้จากรูปที่ 2.5



รูปที่ 2.5 แสดงลักษณะโครงสร้างสำคัญของรหัสแท่งแต่ละแบบ

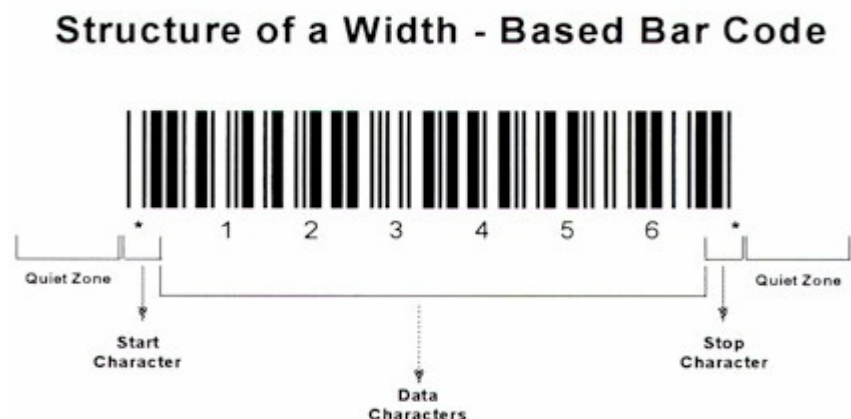
1. รหัสแท่งแบบหนึ่งมิติ (1D-Barcode) หรือ Linear barcodes

รหัสแท่งแบบหนึ่งมิติ (1D-Barcode) หรือ Linear barcodes จะมีลักษณะเป็นแถบเล็กๆ หลายแถบ เรียงกันในแนวดิ่ง โดยจำนวนแถบบ่อยมากก็ยิ่งบรรจุข้อมูลได้มากขึ้น แต่ก็ต้องใช้พื้นที่มากขึ้นเช่นเดียวกัน และจะทำให้การสแกน มีความยากขึ้นตามไปด้วย

รหัสแท่งแบบหนึ่งมิติ (1D-Barcode) ชนิด รหัส 39

รหัส 39 เป็นรหัสชนิดแรกที่ใช้แทนตัวอักษรด้วย ปัจจุบันได้มีรหัสซึ่งขยายจากรหัส 39 แล้วคือรหัส 128 รหัส 39 นั้นประกอบด้วยสัญลักษณ์ 43 ตัว (เดิมมี 39 ตัว) ซึ่งแบ่งเป็นพยัญชนะ 26 ตัว ตัวเลข 10 ตัว และอักษรพิเศษที่เหลือ รหัส 39 นี้สามารถถือเป็นรหัส 3 ใน 9 เพราะหนึ่งตัวรหัสประกอบด้วย 9 ตัวประกอบ โดยสามตัวในนั้นจะเป็นแถบกว้าง และอีกสองตัวจะเป็นแถบแคบ หนึ่งแถบรหัสจะมีหนึ่งถึงสามตัวอักษรเท่านั้นซึ่งตามด้วย Check digit ดังนั้นรหัส 39 จึงมีความแน่นอนในการอ่านสูงแต่เปลืองเนื้อที่มาก รหัสชนิดนี้มีใช้กันมากในอุตสาหกรรมอิเล็กทรอนิกส์ โดยใช้ในการแยกชนิดแผงวงจร

รหัส 39 จะมีส่วนประกอบที่สำคัญอยู่ 4 ส่วนคือ รหัสเริ่มต้นรหัสสิ้นสุด รหัสข้อมูล รหัสตรวจสอบและบริเวณที่ว่างเปล่า โดยมีลักษณะโครงสร้างดังรูปที่ 2.6



รูปที่ 2.6 แสดงลักษณะโครงสร้างสำคัญของ Barcode รหัส 39

ส่วนประกอบที่สำคัญของรหัสแท่งแบบหนึ่งมิติ ได้แก่

Quiet Zone เป็นบริเวณที่ว่างเปล่า ไม่มีการพิมพ์อะไรทั้งสิ้นจะมีอยู่ก่อนและหลังรหัสแท่ง **Start/Stop Character** เป็นบริเวณแถบแท่ง และช่องว่างเพื่อเป็นจุดบอกการเริ่มต้น หรือจุดจบของข้อมูล

Data เป็นบริเวณแถบแท่ง และช่องว่าง ที่แทนข้อมูลต่างๆที่เราต้องการ

Check Digit เป็นบริเวณแถบแท่ง และช่องว่างในที่ใช้ตรวจสอบในส่วนของข้อมูลเพื่อให้มั่นใจว่าข้อมูลที่อ่านได้นั้นถูกต้อง

2. รหัสแท่งแบบสองมิติ (2D-Barcode)

รหัสแท่งแบบสองมิติ หรือที่เรียกว่า Two Dimension Barcode โดยในโครงงานฉบับนี้เราจะเน้นในส่วนของรหัสแท่งชนิดนี้เป็นหลัก ซึ่งมีลักษณะแตกต่างจากบาร์โค้ดทั่วไป กล่าวคือมีลักษณะเป็นแบบสองมิติ มีแถบรหัสผสมกันทั้งแนวตั้งและแนวนอน ซึ่งลักษณะการเรียงตัวทั้งสองแกนนี้เองที่ทำให้มันมีรูปแบบที่แตกต่างกัน มากมายหลายรูปแบบ ทั้งยังมีขนาดพื้นที่ที่เล็กกว่าบาร์โค้ดแบบ 1 มิติ แต่สามารถบรรจุข้อมูลได้มากกว่าหลายเท่า เป็นผลทำให้เครื่องสแกนที่ใช้มีขนาดเล็กลงมาก ทั้งยังมีความรวดเร็วและง่ายต่อการสแกนมากขึ้นด้วย

รหัสแท่งแบบสองมิติ (2D-Barcode) ชนิด คาด้าเมทริกซ์

ดาต้าเมทริกซ์ เป็นรหัสแท่งแบบสองมิติ ที่ประกอบด้วยโครงสร้างรูปสี่เหลี่ยมจัตุรัสสีขาว และดำที่ใช้แทนส่วนของบิตข้อมูล 0 กับ 1 โดยมีการจัดเรียงกันเป็นรูปสี่เหลี่ยมจัตุรัสและสี่เหลี่ยมผืนผ้า ข้อมูลที่นำมาเข้ารหัสสามารถเป็นได้ทั้ง text หรือข้อมูลดิจิทัลก็ได้ ขนาดของข้อมูลโดยทั่วไปมีตั้งแต่ไม่กี่ไบต์จนถึง 2 K ไบต์ ในส่วนของความยาวข้อมูลที่ได้มีการเข้ารหัสแล้วจะขึ้นอยู่กับอัลกอริทึมที่ใช้ในการสร้างภาพรหัสแท่งนั้น

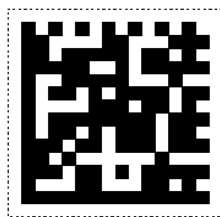
ลักษณะสำคัญคือได้มีการเพิ่มส่วนของ Error Correction Code เข้าไปในตัวดาต้าเมทริกซ์ เพื่อให้ข้อมูลมีความแข็งแกร่งขึ้น กล่าวคือหากมีส่วนใดส่วนหนึ่งของข้อมูลเกิดความเสียหายก็ยังสามารถอ่านข้อมูลทั้งหมดได้อย่างถูกต้องทั้งหมด ความสามารถในการเก็บตัวข้อมูลสามารถเก็บได้สูงสุดถึง 2335 Alphanumeric Characters ขนาดของตัว Data Matrix เป็น Matrix ขนาด 10×10 ถึง 144×144 ในแบบของรูปสี่เหลี่ยมจัตุรัส และ เป็น Matrix ขนาด 8×18 ถึง 16×48 ในแบบของรูปสี่เหลี่ยมผืนผ้า

ECC200 เป็นเวอร์ชันล่าสุดของ Data Matrix ทั้งยังสนับสนุนการทำ error checking และ correction algorithms (เช่น Reed-Solomon) เป็นอัลกอริทึมที่ใช้ในการตรวจสอบและแก้ไขความผิดพลาดของข้อมูลสำหรับรหัสแท่งซึ่งจะยังสามารถอ่านข้อมูลได้ถึง 25% ของข้อมูลที่เกิดความผิดพลาดไป

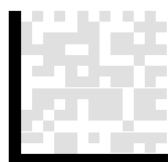
ดาต้าเมทริกซ์ถูกสร้างขึ้นโดย RVSI ปัจจุบันได้ครอบคลุมถึงมาตรฐาน ISO ต่างๆ ดังต่อไปนี้ และ อนุญาตให้สามารถใช้ได้โดยทั่วไป

- ISO/IEC 15418:1999 – Symbol Data Format Semantics
- ISO/IEC 15434:1999 – Symbol Data Format Syntax
- ISO/IEC 15415 – 2-D Print Quality Standard

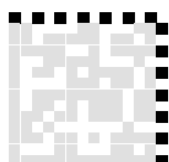
ดาต้าเมทริกซ์มีส่วนประกอบที่สำคัญอยู่ 4 ส่วนคือ Solid Border, Broken Border, Data Storage และ Quiet Zone โดยมีลักษณะโครงสร้างดังรูปที่ 2.7



Data Matrix Symbol Shown Complete



Solid Border



Broken Border



Data Storage

รูปที่ 2.7 แสดงลักษณะ โครงสร้างสำคัญของ Barcode ชนิด Data Matrix

ส่วนประกอบที่สำคัญของรหัสแท่งแบบสองมิติ ได้แก่

L-Shaped หรือ **Solid Border** เป็นแถบรูปตัวแอลที่ใช้ในการอ้างตำแหน่งและมุม

Broken Border เป็นส่วนที่ใช้ในการบอกขอบเขตสิ้นสุดของข้อมูล ในส่วนขอบมุมด้านบน

Data Storage บริเวณแถบแท่ง และช่องว่างที่แทนข้อมูลต่างๆที่เราต้องการ

Quiet Zone เป็นบริเวณที่ว่างเปล่าไม่มีการพิมพ์อะไรทั้งสี่ด้านจะมีอยู่ก่อนและหลังรหัสแท่ง

2.3 จาวาคืออะไรและสามารถทำอะไรได้บ้าง

2.3.1 ภาพรวมของจาวา

ภาษาจาวาถูกพัฒนาขึ้นโดยบริษัท ซันไมโครซิสเต็มส์ (Sun Microsystems Inc.) เป็นภาษาสำหรับเขียนโปรแกรมภาษาหนึ่ง มีลักษณะสนับสนุนการเขียนโปรแกรมเชิงวัตถุ (OOP : Object-Oriented Programming) ที่ชัดเจน โปรแกรมต่างๆถูกสร้างภายในคลาสโปรแกรมเหล่านั้นถูกเรียกว่า เมทอด (method) หรือ พฤติกรรม (behavior) โดยปกติจะเรียกแต่ละคลาส (Class) ว่าอ็อบเจกต์ (Object) โดยแต่ละอ็อบเจกต์ มีพฤติกรรมมากมาย โปรแกรมที่สมบูรณ์จะเกิดจากหลายอ็อบเจกต์หรือหลายคลาสมารวมกัน โดยแต่ละคลาสจะมีเมทอดหรือพฤติกรรมแตกต่างกันไป

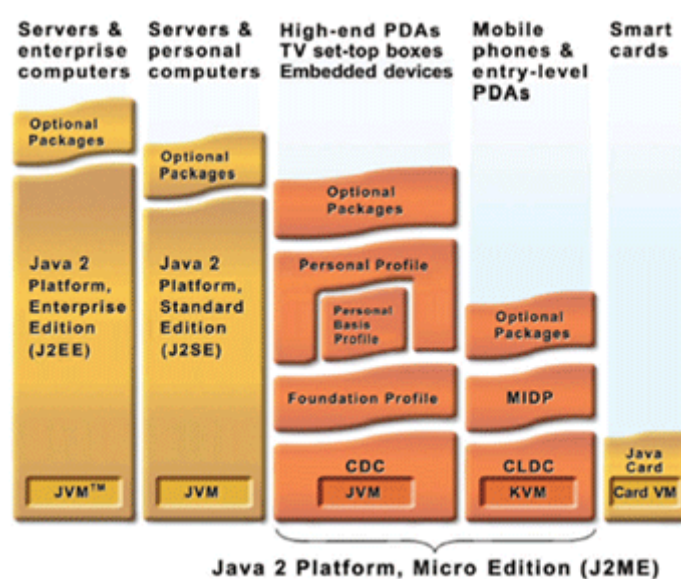
2.3.2 แพลตฟอร์มของจาวา

แพลตฟอร์มจาวา2 ที่ใช้งานอยู่ในขณะนี้ มี 3 รุ่นด้วยกัน แต่ละรุ่นเน้นนำไปใช้กับเฉพาะกลุ่มแอปพลิเคชัน ดังนี้

J2EE: Java 2 Platform, Enterprise Edition (Enterprise/Server) ใช้งานกับแอปพลิเคชันบนเครื่องเซิร์ฟเวอร์สำหรับองค์กรที่รองรับระบบงานใหญ่ๆ และไคลเอนต์จำนวนมาก

J2SE: Java 2 Platform, Standard Edition (Core/Desktop) ใช้งานกับแอปพลิเคชันบนเครื่องคอมพิวเตอร์แบบตั้งโต๊ะทั่วไป

J2ME: Java 2 Platform, Micro Edition (Mobile/Wireless) ใช้งานกับแอปพลิเคชันรุ่นใหม่ซึ่งเน้นกลุ่มผู้ใช้อุปกรณ์อิเล็กทรอนิกส์และอุปกรณ์ฝังตัว (Embedded)



(จาก <http://www.sourcecode.in.th/j2me>)

รูปที่ 2.8 แสดงแพลตฟอร์มจาวา 2 ทั้ง 3 รุ่นที่ใช้งานอยู่ในขณะนี้

2.3.3 J2ME คืออะไร

J2ME หรือเรียกกันเต็มๆ ว่า Java 2 Micro Edition นั้นก็คือ Edition หนึ่งของ JAVA2 ที่ทำมาเพื่อสำหรับการใช้งานบนอุปกรณ์ขนาดเล็กๆ ซึ่งมีทรัพยากรจำกัด เช่นขนาดของหน่วยความจำและความสามารถในการประมวลผลเป็นต้น อุปกรณ์เหล่านั้นก็ได้แก่พวก handheld device เช่น Palm, Pocket PC หรือ โทรศัพท์มือถือ เป็นต้น

ในแต่ละ Edition ของ JAVA นั้นก็จะมี Virtual Machine เป็นของตัวเองซึ่งที่ต้องแตกต่างกันก็เพราะเพื่อรองรับงานคนละแบบคนละขนาดนั่นเอง จากรูปก็จะเห็นได้ว่า Hotspot VM จะเป็น Java Virtual Machine สำหรับงานที่เป็นลักษณะ Server Side Application ของ J2EE นั่นเอง

หรือจะเป็น JVM ซึ่งเหมาะสำหรับงานซึ่งทำงานบน Desktop ของ J2SE ซึ่งจริง ๆ แล้ว Hotspot VM เองก็สามารถใช้กับ J2SE ได้เช่นกัน

ในส่วน Virtual Machine (VM) สำหรับ J2ME จะมีสอง VM คือ C Virtual Machine (CVM) กับ K Virtual Machine (KVM) ซึ่งเป็น VM ที่มีขนาดกะทัดรัดหรือขนาดเล็กและใช้ resources น้อยกว่า Hotspot VM และ JVM มาก

ในแต่ละ Edition ของ JAVA จะ Define ส่วนประกอบหลัก ๆ ซึ่งมี 3 ส่วนคือ

1. Java Virtual Machine ซึ่งจะอยู่ในตัว Device
2. Libraries และ APIs สำหรับ Device แต่ละชนิด
3. Development tools และ Device Configurations

2.3.4 คอนฟิกูเรชันใน J2ME

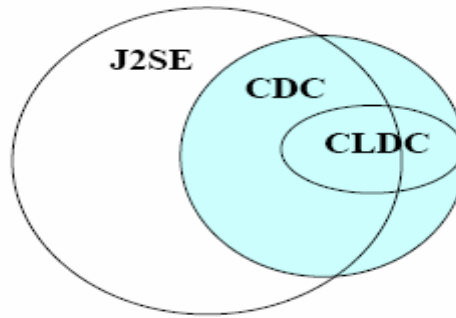
Configuration และ Profiles ถือเป็นส่วนหลักสำคัญของ J2ME ซึ่งจุดประสงค์ของการกำหนดสองส่วนนี้ก็เพื่อให้ได้ virtual machines และ class libraries ที่เหมาะสมสำหรับ devices แต่ละกลุ่มประเภท

Configuration หมายถึงชุดของ feature ของ Java Virtual Machine และ Java Class Library สำหรับ device ในแต่ละประเภทหรือในแต่ละ horizontal market ใดๆ หรือพูดอีกนัยหนึ่ง Configuration จะเป็นการกำหนด feature หรือ library มาตรฐานซึ่งจะมีเหมือนกันในทุก ๆ device ที่จัดอยู่ในกลุ่มประเภทเดียวกัน

ปัจจุบันได้แบ่ง Configuration สำหรับ J2ME ออกเป็น 2 ประเภทคือ CDC (Connected Device Configuration) และ CLDC (Connected Limited Device Configuration) โดยการจัดกลุ่มจัดโดยแบ่งตาม device ซึ่งมีความคล้ายกันในเรื่องของขนาดของ memory และ processing power

CDC devices เป็นอุปกรณ์ประเภท Shared, fixed, connected information devices ซึ่งมี user interface ขนาดใหญ่ มี memory 2 - 16 Mb และมี CPU 32 bit เป็นอย่างน้อยและใช้ TCP/IP ตัวอย่างของ device พวกนี้ได้แก่ TV set-top box, Internet TV, Internet-enabled screen phones, high-end communicators and car entertainment/navigation systems

CLDC devices เป็นอุปกรณ์ประเภท Shared, fixed, connected information devices ซึ่งมี user interface แบบง่ายๆ ขนาดเล็กมี memory 128 - 1 Mb และมี CPU 16,32 bit และมักไม่ใช้ TCP/IP network ตัวอย่างของ device พวกนี้ได้แก่ low-end cell phone, two-way pager และ PalmOS handhelds จากรูปที่ 2.10 จะเห็นว่า CLDC จะเป็น Subset ของ CDC



รูปที่ 2.9 แสดงความสัมพันธ์ระหว่าง J2SE Configuration และ J2ME

จากรูปจะเห็นว่า Function ต่าง ๆ ของ CDC และ CLDC ส่วนใหญ่จะต้องถูกสืบทอดมาจาก J2SE ส่วนที่เกินออกมาก็ทำเพื่อใช้สำหรับ small device

2.3.5 Mobile Information Device Profile (MIDP)

เมื่อพูดถึง MIDP ก็จะหมายถึงประเภทของ Device พวกที่มีคุณสมบัติต่อไปนี้เช่น small display มี touch screen หรือ keypad สามารถ connect mobile network ด้วย bandwidth ที่จำกัด

MIDP ประกอบด้วย APIs ที่ทำหน้าที่ดังต่อไปนี้

- User Interface จัดการเกี่ยวกับการแสดงผล
- Persistent storage จัดการเกี่ยวกับการเก็บข้อมูลและฐานข้อมูล
- Networks จัดการเกี่ยวกับการเชื่อมต่อเน็ตเวิร์ค
- Application life-cycle จัดการเกี่ยวกับลำดับขั้นตอนการทำงาน
- Event handling จัดการเกี่ยวกับอีเวนต์ต่างๆ

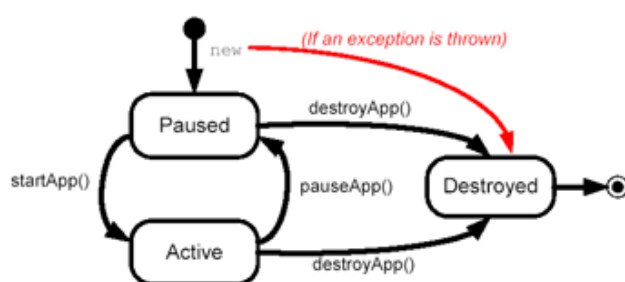
MIDP Packages จะมีอยู่ 3 Packages คือ

- javax.microedition.midlet เป็น API ในการสร้างโปรแกรมหลัก
- javax.microedition.lcdui เป็น API ในการจัดการ User Interface
- javax.microedition.rms RMS (Record Management System) เป็นส่วนของการเก็บข้อมูล เช่นเดียวกับฐานข้อมูล

2.3.6 วงจรการทำงานของ MIDlet

MIDlet หมายถึง แอปพลิเคชันบน MIDP MIDlet มีส่วนคล้ายกับ Java Applet แม้จะไม่มีเมธอด main() แต่ MIDlet ก็นำคลาส javax.microedition.midlet.MIDlet ตลอดจนเมธอดทั้ง 3 ตัว ได้แก่ startApp() pauseApp() และ destroyApp() ซึ่งเป็นสถานะหลักของการทำงานเข้ามาใช้นอกจากนี้ MIDlet ยังกำหนด constructor แบบ public ที่ไม่มีอาร์กิวเมนต์ใดๆ ได้อีกด้วย

ในความเป็นจริงแล้ว การพัฒนา MIDlet จะเป็นในลักษณะเดียวกับการพัฒนา Java Applet คือ Java Applet จะเป็นการสืบทอดจาก class Applet ส่วน MIDP จะเป็นการสืบทอดจาก class MIDlet



(จาก <http://www.sourcecode.in.th/j2me>)

รูปที่ 2.10 แสดงวงจรการทำงานของ MIDlet

เมื่อเราได้ทำการสร้าง class ที่ได้มีการสืบทอดคุณสมบัติมาจาก MIDlet Package ที่ชื่อ javax.microedition.midlet.MIDlet แล้ว class ของเราจะมีวงจรการทำงานเริ่มจากการสร้างหลังจากนั้นก็อยู่ในสถานะ Paused(หยุด) เมื่อเราต้องการให้มีการทำงานเกิดขึ้นเราก็ต้องเรียกใช้ method ที่ชื่อ startApp() สถานะของคลาสก็เปลี่ยนไปเป็นสถานะ Active(ทำงาน)ดังรูปที่ 2.11 และหากต้องการให้ class หยุดการทำงานก็สามารถทำได้โดยการเรียกใช้ Method pauseApp() สุดท้ายก็คือการจบการทำงานโดยเรียกใช้ Method destroyApp() โปรแกรมก็จะเข้าสู่สถานะ Destroyed(ถูกทำลาย)

2.4 เว็บเซอร์วิส (Web Service)

2.4.1 เว็บเซอร์วิสคืออะไร

เว็บเซอร์วิส เป็นระบบซอฟต์แวร์ที่ออกแบบมาเพื่อสนับสนุนการทำงานระหว่างคอมพิวเตอร์กับคอมพิวเตอร์ผ่านระบบเครือข่าย โดยที่ภาษาที่ใช้ในการติดต่อสื่อสารระหว่างคอมพิวเตอร์คือภาษาเอ็กซ์เอ็มแอล (XML)

การอธิบายวิธีการใช้งานของเว็บเซอร์วิสนั้นจะอธิบายโดยใช้ภาษาวิสเดิล WSDL (Web Services Description Language) ซึ่งเป็นภาษา XML ประเภทหนึ่ง

ระบบอื่นๆจะสามารถติดต่อและทำงานกับเว็บเซอร์วิส โดยใช้โปรโตคอลที่ชื่อว่า SOAP (Simple Object Protocol) ซึ่งใช้ภาษา XML เป็นมาตรฐานในการติดต่อระหว่างระบบโดยผ่านทางโปรโตคอลอื่นที่ใช้ในการส่งข้อมูลบนเว็บ อาทิเช่น โปรโตคอล HTTP

2.4.2 พื้นฐานของเว็บเซอร์วิส

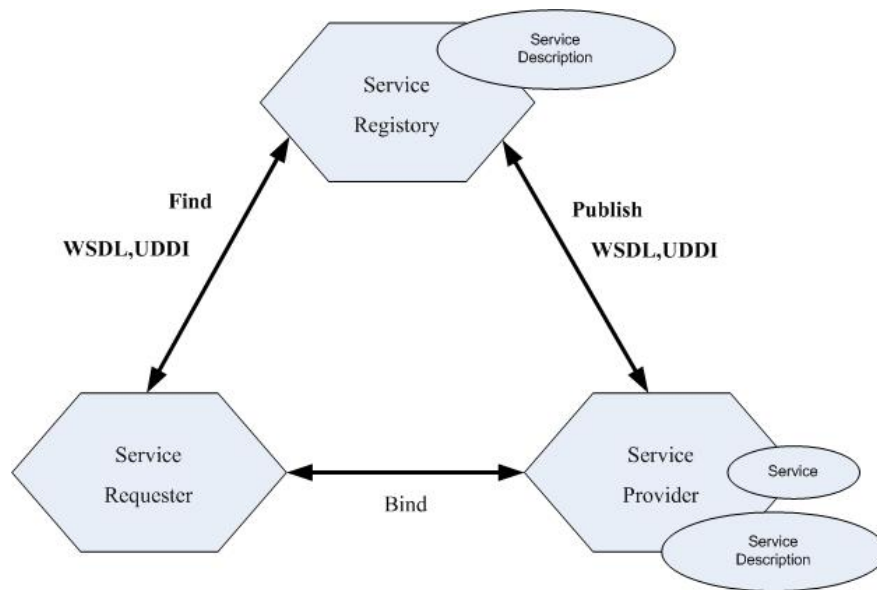
พื้นฐานของ Web Service ก็คือ XML กับ HTTP ซึ่งจะพบว่า HTTP ก็เป็นที่รู้จักกันดีและไปได้ทั่วทุกแห่งที่มี Internet ส่วน XML ก็คือภาษาสากลที่คุณสามารถปรับแต่งได้ตามใจชอบเพื่อให้เกิดกิจกรรมระหว่าง client และบริการ หรือระหว่างส่วนประกอบต่างๆ เบื้องหลัง Web server ก็คือ ข้อความ XML จะถูกแปลงให้การขอบริการจาก Middle ware และผลที่ได้ก็จะแปลงกลับมาในรูป XML

2.4.3 ลักษณะของเว็บเซอร์วิส

การพัฒนาเว็บเซอร์วิสใช้สถาปัตยกรรมบริการในลักษณะที่เรียกว่า สถาปัตยกรรมของแนวคิดทางด้านบริการหรือ เอสโอเอ (Service-Oriented Architecture: SOA) เป็นแนวคิดเบื้องต้น แอปพลิเคชันส่วนใหญ่ในโลกของธุรกิจที่ใช้งานในปัจจุบัน เป็นแอปพลิเคชันและระบบย่อยที่ถูกสร้างขึ้น มีการทำงานที่ต้องสัมพันธ์กันอย่างไม่เป็นอิสระจากกัน การเปลี่ยนแปลงการทำงานในระบบย่อยหรือแอปพลิเคชันหนึ่ง อาจจะมีผลกระทบกับอีกแอปพลิเคชันหนึ่ง หรือบางครั้งอาจจะกระทบทั้งระบบ ทำให้การบำรุงรักษานั้นทำได้ยาก และมีต้นทุนที่สูงขึ้น รวมทั้งยังเป็นข้อจำกัดในการเชื่อมต่อกับระบบของคู่ค้าอื่นๆ

Service-Oriented Architecture มีส่วนประกอบหลัก 3 ส่วนดังนี้

1. Requestor คือใครก็ตามที่ต้องการเรียกใช้บริการจาก Provider ซึ่งสามารถค้นหาบริการที่ต้องการได้จาก UDDI registry หรือ Service Registry หรือติดต่อจาก Provider โดยตรง
2. Registry ทำหน้าที่เป็นตัวกลางให้ Provider มาลงทะเบียนไว้ โดยใช้ WSDL ไฟล์ บอกรายละเอียดของบริษัทและบริการที่มีให้ ซึ่งอาจจะใช้หรือไม่ใช้ก็ได้
3. Provider เป็นผู้ให้บริการ มีหน้าที่ในการเปิดบริการเพื่อรองรับการขอใช้บริการจาก Requestor ที่เรียกเข้ามาขอใช้



รูปที่ 2.11 โมเดลของ SOA ซึ่งแสดงความสัมพันธ์ของส่วนประกอบหลักในแต่ละส่วน

2.4.4 เทคโนโลยีที่ใช้ในการพัฒนาเว็บเซอร์วิส

XML (The Extensible Markup Language 1.0)

XML (The Extensible Markup Language 1.0) เป็นภาษา Markup ที่เป็น text-based ซึ่งทำให้เป็นมาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตอย่างรวดเร็ว ผู้ที่ทำหน้าที่รับผิดชอบและกำหนดมาตรฐานของ XML คือ World Wide Web Consortium (W3C) ความแตกต่างระหว่าง XML กับ HTML คือ HTML ถูกนำมาใช้ในการสร้างเว็บเพจที่สามารถแสดงผลได้โดยโปรแกรมบราวเซอร์ แต่ XML จะใส่แท็กได้อย่างอิสระแล้วทำการส่ง XML ชุดนี้ไปประมวลผลยังแอปพลิเคชันใดๆที่สามารถใช้ข้อมูลใน XML นี้ได้

XML เป็นภาษาที่มีลักษณะเป็นแท็กคล้าย HTML แต่ไม่ได้มุ่งที่การแสดงผล XML มุ่งที่การสื่อความหมายโดยอนุญาตให้ผู้ใช้สามารถกำหนดแท็กขึ้นได้เองเพื่อให้สื่อความหมายทางภาษาของมนุษย์ แต่คอมพิวเตอร์เองก็เข้าใจเช่นกันทำให้ข้อมูลระหว่างแท็กสามารถนำไปประมวลผลต่อได้

SOAP (Simple Object Access Protocol)

SOAP กลายเป็นสิ่งที่มีความสำคัญสำหรับ Web Services อย่างรวดเร็วเป็นโปรโตคอลที่ผู้จัดทำ Web Services เลือกใช้ที่จะส่ง message ระหว่าง Web Services SOAP เป็น Transport Protocol ที่มี XML เป็นพื้นฐานและใช้ HTTP เป็นโปรโตคอลร่วมในการส่งผ่านเครือข่าย SOAP จะระบุวิธีในการเข้ารหัสส่วนหัว (Header Encoding) ของทั้ง HTTP และไฟล์ XML ให้อย่างชัดเจนทั้งในส่วนของการติดต่อไปยังคอมพิวเตอร์อีกเครื่องหนึ่งและส่งผ่านข้อมูลไปให้รวมถึงวิธีที่โปรแกรมซึ่งถูกเรียกนั้นจะส่งค่ากลับคืนมาด้วย

SOAP ได้ถูกกำหนดให้เป็น Services-Oriented Architecture Protocol เรียบร้อยแล้ว เนื่องจากมันได้ถูกใช้ในการพัฒนา SOA อย่างแพร่หลายแล้วนั่นเอง จุดเด่นของ SOAP ก็คือเป็น โปรโตคอลที่เป็นกลางกล่าวคือ ไม่มีใครเป็นเจ้าของและเป็นโปรโตคอลที่ทำงานกับโปรโตคอลอื่น หลายชนิด การพัฒนาก่อนุญาตให้ทำได้อย่างอิสระตามแพลตฟอร์มระบบปฏิบัติการแบบจำลอง วัตถุ (Object model) และภาษาโปรแกรมของผู้ที่ทำการพัฒนา

WSDL (Web Services Description Language)

WSDL (Web Services Description Language) เป็นภาษาที่ใช้อธิบายคุณลักษณะการใช้ บริการของ Web Services และวิธีการติดต่อกับ Web Services ความต้องการของนิยามนี้มีเกี่ยวเนื่อง กับความต้องการของ distributed system ที่จะมาทำการกำหนด Interface Definition Language(IDL) โดยใช้ภาษา XML, WSDL เกิดจากการรวมแนวคิดของ NASSL (The Network Accessible Service Specification Language), WDS (Well-Defined Services) ของบริษัทไอบีเอ็ม SDL (The Service Description Language) และ SCL (the SOAP Contract Language) ของบริษัท ไมโครซอฟท์ ปัจจุบัน WSDL เป็นภาษาที่อยู่ในการดูแลของ W3C (World Wide Web Consortium) ซึ่งยังไม่เป็นมาตรฐานที่สมบูรณ์ เวอร์ชันที่ใช้งานอยู่ในปัจจุบันคือ WSDL 1.1